SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

Team 166_EyeScream (Domain 1)

Table of Contents

1. Introduction....3

1.1 Purpose....3

1.2 Background....4

1.3 Target Stakeholder....5

2. System Architecture & Design....7

2.1 High Level Architecture....7

2.2 Technological Stack....14

2.3 Key Data Flows....16

3. Functional Requirements & Scope....18

3.1 Minimum Viable Product:....18

3.2 Non-Functional Requirements (NFRs)....20

3.3 Out of Scope / Future Enhancements....21

4. Monitor, Evaluation, Assumptions & Dependencies....22

4.1 Technical Evaluation:....22

4.2 Monitoring....24

4.3 Assumptions....24

4.4 External Dependencies....25

5. Project Management & Team Contributions....26

5.1 Project Timeline....26

5.2 Team Members Role....27

5.3 Recommendations....28

1. Introduction

TaxMate is an AI-powered tax compliance platform designed for Malaysian SMEs. It addresses the critical pain point that small and medium business owners face in managing their tax reports, such as bi-monthly SST-02 filing obligations. This process is traditionally manual, error-prone, and time-consuming. TaxMate automates the entire compliance pipeline from receipt/invoice upload, through OCR extraction and AI-driven tax classification, to an accountant review and final tax report form generation with a digital signature. All governed by an intelligent workflow orchestrator powered by Z.AI GLM.

1.1 Purpose

The system analysis document highlights the technical scope and design-related decisions behind the TaxMate development. The key elements of the system that have been covered are as follows:

1. **Architecture:** TaxMate is a client-server web application with a decoupled frontend-backend architecture. The backend is built as a modular monolith using FastAPI, structured around domain-driven modules (agents, services, routers, models). It is deployed via Docker Compose with a PostgreSQL 16 database container. The system communicates with Z.AI GLM API (via ILMU gateway) as the core LLM reasoning engine for multi-step agentic tax analysis.

2. **Data Flows:** A business owner uploads a receipt image (JPG/PNG) or PDF via the React frontend. The file is sent as multipart/form-data to the FastAPI backend. The backend runs PaddleOCR to extract raw text, then passes the OCR text to the GLM-powered Tax Agent, which executes 3-8 tool calls (lookup_sst_rule, validate_sst_number, calculate_tax, flag_for_human_review, generate_tax_advice) before producing a structured JSON classification. The result persisted in PostgreSQL and returned to the frontend.

3. **Model Process:** The model shows the end-to-end workflow: (1) Business Owner uploads receipts and receives AI classification with risk flags; (2) Accountant reviews flagged

documents, edits fields, approves/rejects; (3) Accountant digitally signs the tax report form; (4) System generates the official PDF, emails it to the owner, and marks the filing period complete. The AI Orchestrator monitors all stages and provides adaptive recommendations.

4. **Role of Reference:** This document is a reference for all developers, testers, and hackathon evaluators to understand the overall high-level architecture, GLM integration patterns, data flows, and system design decisions.

1.2 Background

Malaysian SMEs with annual turnover exceeding RM 500,000 (RM 1.5M for F&B;) are mandated to register for SST and file bi-monthly SST-02 returns. The existing process is heavily manual: business owners collect physical receipts, accountants manually review each one, compute tax amounts, and fill out government PDF forms. E-Invoice Phase 4 (effective Jan 2026) for businesses with RM 1M-5M revenue further increases compliance burden with penalties of RM 200-20,000 per non-compliant invoice starting Jan 2027.

Previous Version: Before TaxMate, SMEs relied on spreadsheets and manual data entry. Accountants reviewed each receipt individually without AI assistance, with no automated cross-document duplicate detection, anomaly flagging, or tax calculation verification. All the tax report forms, including the SST-02 form, were filled manually in the JKDM portal.

Changes in Major Architectural Components

- Introduction of a multi-step agentic AI pipeline powered by Z.AI GLM that autonomously calls tools (rule lookup, SST validation, tax calculation, cross-verification) before classification.
- Addition of a Workflow Orchestrator agent that maintains state across 7 compliance stages (INTAKE →ANALYSIS → REVIEW_PREP → AWAITING_REVIEW → FILING →

NOTIFICATION → COMPLETE) and makes adaptive decisions based on risk thresholds, confidence scores, and filing deadlines.

- Cross-document batch analysis (L4 agent) that performs duplicate detection, anomaly spotting, and tax optimization across all receipts in a filing period.

New capabilities that are only introduced in this

- AI-powered OCR + classification with 5 specialised tool calls per document
- Automated SST number format validation against JKDM standards
- Real-time tax calculation cross-verification against receipt-stated SST amounts
- Cross-document duplicate and anomaly detection
- Digital signature with PDF form filling on the official SST-02 template
- Automated email notification with AI-personalised content
- AI Copilot chat interface for both owners and accountants
- Adaptive workflow orchestration with risk-based gating
- Auto-approve the receipt/invoice if the confidence is greater than or equal then 90%
- Automatically match the owner with an accountant who has extensive experience in this field or specify a dedicated accountant when the owner registers.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
Business Owner (Client)	Uploads receipt images/PDFs, reviews AI classifications,	Simple drag-and-drop upload, accurate AI extraction, clear risk

	downloads signed tax report form (SST-02 forms), asks AI Copilot questions, browse tax insight generated by GLM	flags, downloadable official SST-02 PDF, AI chatbot for tax questions, clear receipt/invoice summary and tax insight
Accountant	Reviews AI-flagged documents in Workbench, edits extracted fields, approves/rejects documents, digitally signs tax report form (SST-02), asks AI Copilot questions, and browses efficiency summary	Queue-based review workflow, AI auto-approval for high-confidence receipts (≥90%), editable fields, AI briefing summaries, AI chatbot for tax questions, clear efficiency summary
Development Team	Build and maintain the TaxMate platform	Clear API contracts, modular codebase separated by domain, well-documented architecture
QA Team	Validating the system behavior for all the user roles	Defined scope, request/response formats, edge case coverage (invalid SST numbers, OCR failures, rate limits)

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System	Web Application
Architecture	Modular Monolith with decoupled Frontend-Backend, Docker Compose deployment

TaxMate is structured as a client-server system with two role-based interfaces served from a single React SPA:

(1) Business Owner Interface: dashboard, upload, document list, tax insights, downloads, chatbot, profile;

(2) Accountant Interface: client list, review workbench with queue-based document processing, digital signature pad, efficiency summary, profile;

Both clients communicate with a single FastAPI backend through RESTful API endpoints. The backend integrates with Z.AI GLM API for all LLM-powered reasoning. PostgreSQL 16 handles persistent storage. PaddleOCR runs locally for text extraction. The official tax report form (SST-02 PDF) template is filled programmatically using pypdf + ReportLab.

2.1.2 LLM as Service Layer

TaxMate integrates Z.AI GLM (ilmu-glm-5.1) as a service layer through three distinct agent patterns, not a single generic "AI Module". The LLM API is accessed via an Anthropic-compatible SDK through the ILMU gateway (api.ilmu.ai/anthropic), enabling function calling capabilities:

Agent 1: Single-Document Tax Agent (tax_agent.py)

- Purpose: Multi-step agentic classification of individual receipts/invoices

- Pattern: Tool-use loop with function calling — LLM autonomously calls 3-6 tools per document
- Tools: lookup_sst_rule, validate_sst_number, calculate_tax, flag_for_human_review, generate_tax_advice
- Prompt: System prompt embeds Malaysian SST context (tax rates, thresholds, e-invoice rules) + adaptive workflow rules
- Context window: System prompt (~1500 tokens) + OCR text (100-800 tokens) + tool call history (~200-500 tokens/step)
- Token limit: Max 8 agent steps per document. Response parsed as strict JSON with _clean_json() and _repair_json() fallbacks

Agent 2: Cross-Document Batch Analyzer (tax_agent.py → analyze_monthly_batch())

- Purpose: Analyzes ALL documents in a filing period for duplicates, anomalies, compliance risks, tax optimization
- Pattern: Single-prompt structured output — all document summaries in one prompt, specific JSON schema response
- Output: batch_health, totals, duplicate_warnings, anomalies, tax_tips, compliance_issues, filing_readiness

Agent 3: Workflow Orchestrator (workflow_orchestrator.py)

- Purpose: Stateful pipeline management — decides what action to take next based on document states, risk levels, filing deadlines
- Pattern: Tool-use loop with 7 orchestrator tools: check_workflow_status, trigger_batch_analysis, prepare_accountant_brief, generate_filing, send_notification, assess_risk_level, recommend_next_action

- Adaptive rules: If >30% risk flags → block filing; deadline 0.9 → fast-track; total payable >RM 10K → extra verification
- Fallback: Deterministic rule-based recommendation engine if GLM API fails

Additional LLM-powered features

- generate_sst02_field_mapping() — AI decides how documents map to SST-02 B1 rows and tax rate fields
- generate_review_brief() — AI summarizes document batch for accountant review briefing
- generate_personalized_email() — AI writes owner notification emails based on monthly data • answer_accountant_question() — Context-enriched chat over current month's documents

2.1.3 Dependency Diagram

The following diagram shows how all system components interact. Key API calls are numbered:

1. React Frontend → FastAPI Backend: Axios HTTP with JWT Bearer token (REST JSON/Multipart)

2. FastAPI → Z.AI GLM API: Anthropic-compatible function calling via client.messages.create(). Base URL: api.ilmu.ai/anthropic. Model: ilmu-glm-5.1. Timeout: 180s. Max retries: 2.

3. FastAPI → PostgreSQL: SQLAlchemy ORM queries for CRUD on users and documents tables

4. PaddleOCR: Local in-process ML library, no network dependency. Loaded lazily on first use.

5. FastAPI → SMTP: fastapi-mail sends approved SST-02 PDFs via configurable SMTP with STARTTLS

6. Vite Dev Server → FastAPI: Proxy configuration for /api routes (5173 → 8000)

Token enforcement and input chunking:

- Tax Agent: max 8 steps/document. Tool results truncated to 500 chars in thinking_steps log.
- Batch analysis: all document summaries in single prompt; limited to 200 documents max.
- Workflow Orchestrator: max 5 steps per invocation.
- All LLM calls: max_tokens=4096 (agents), max_tokens=2048 (text generation).
- Rate limit retry: exponential backoff via Tenacity (attempts: 3, wait: 2-10s).

2.1.4 Sequence Diagram

2.1.4.1 Receipt Upload & Classification Flow

The following sequence describes the complete receipt processing pipeline. The Mermaid source is provided alongside for rendering in documentation tools:

Step 1: Owner selects file → Frontend sends POST /api/documents/upload (multipart/form-data)

Step 2: Backend receives file → calls PaddleOCR perform_ocr() → extracts raw text from image/PDF

Step 3: Backend passes OCR text to Tax Agent process_receipt() → Agent enters multi-step loop

Step 4: Agent Step 1 — calls lookup_sst_rule to check relevant SST rules for document type

Step 5: Agent Step 2 — calls validate_sst_number if SST number found on receipt

Step 6: Agent Step 3 — calls calculate_tax to cross-check stated SST against calculated amount

Step 7: Agent Step 4-5 — calls flag_for_human_review (if needed) + generate_tax_advice

Step 8: Agent produces final structured JSON with doc_type, supplier, amount, tax_treatment, confidence, risk_flags, reasoning

Step 9: Backend stores result in PostgreSQL (INSERT document with agent_result JSON)

Step 10: Backend returns ProcessResponse to Frontend → Owner sees classification result

Step 11: Owner clicks "What should I do next?" → Frontend calls POST /api/documents/workflow/next-step

Step 12: Workflow Orchestrator checks state → calls tools (check_status, assess_risk) → returns recommendation

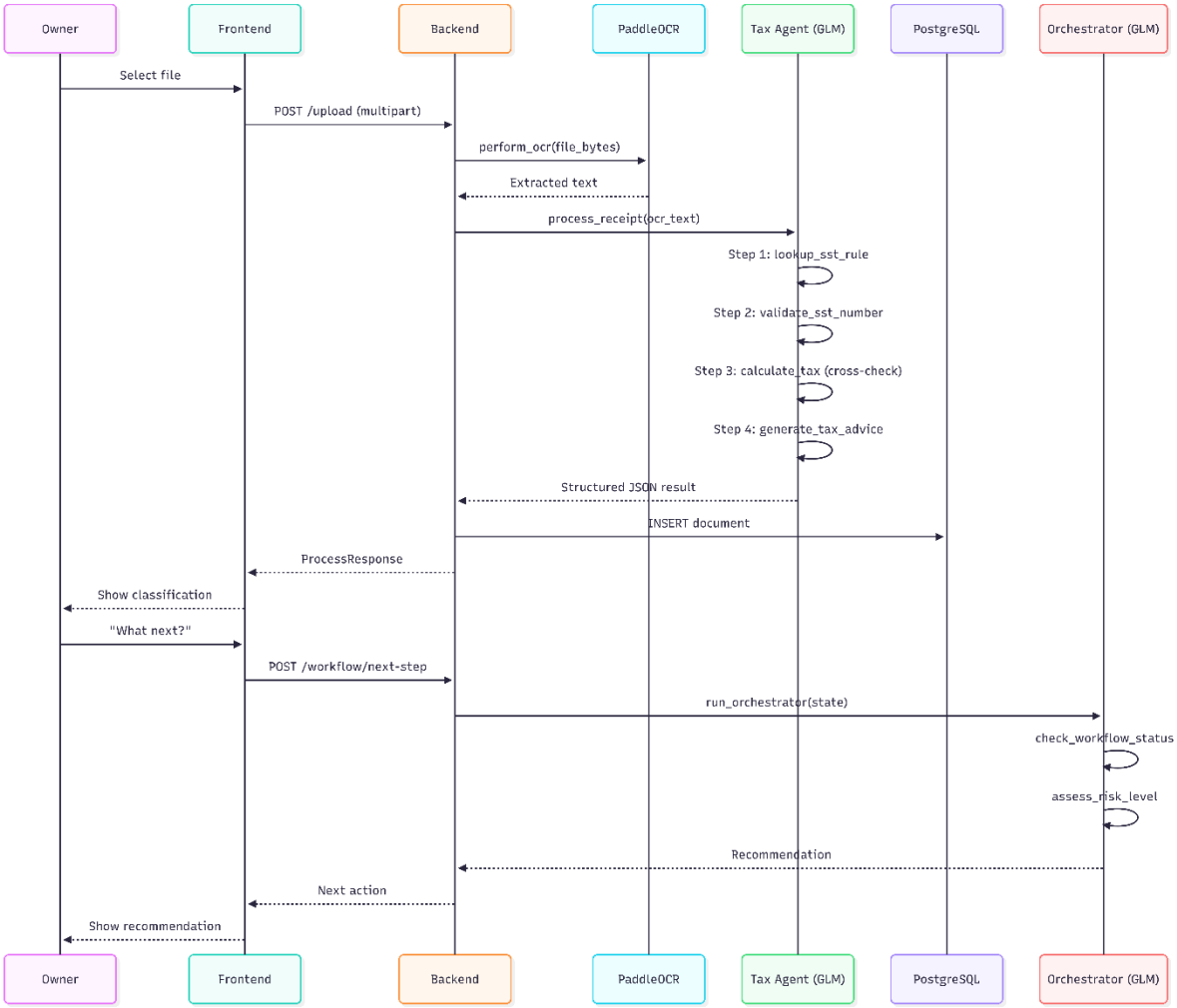

Figure 1: Receipt Upload & Classification Flow Sequence Diagram

2.1.4.2 Accountant Review & SST-02 Signing Flow

Step 1: Accountant opens Workbench → Frontend calls GET /api/documents?status=pending_review

Step 2: Accountant requests AI Briefing → GET /api/documents/review-brief → GLM generates summary

Step 3: Accountant reviews each document → edits fields if needed → PUT /{id}/review (approve/reject)

Step 4: Accountant signs SST-02 → POST /api/documents/sign-sst02 with signature_b64 data

Step 5: Backend calls generate_sst02_field_mapping() → AI decides B1 row mappings and tax rates

Step 6: Backend fills official JKDM SST-02 PDF (pypdf AcroForm) + overlays digital signature (ReportLab)

Step 7: Backend updates all documents to status=signed, stores signature metadata

Step 8: Backend sends email via SMTP (BackgroundTasks) with signed PDF + AI-personalized message

Step 9: Frontend returns signed PDF as base64 → Accountant downloads

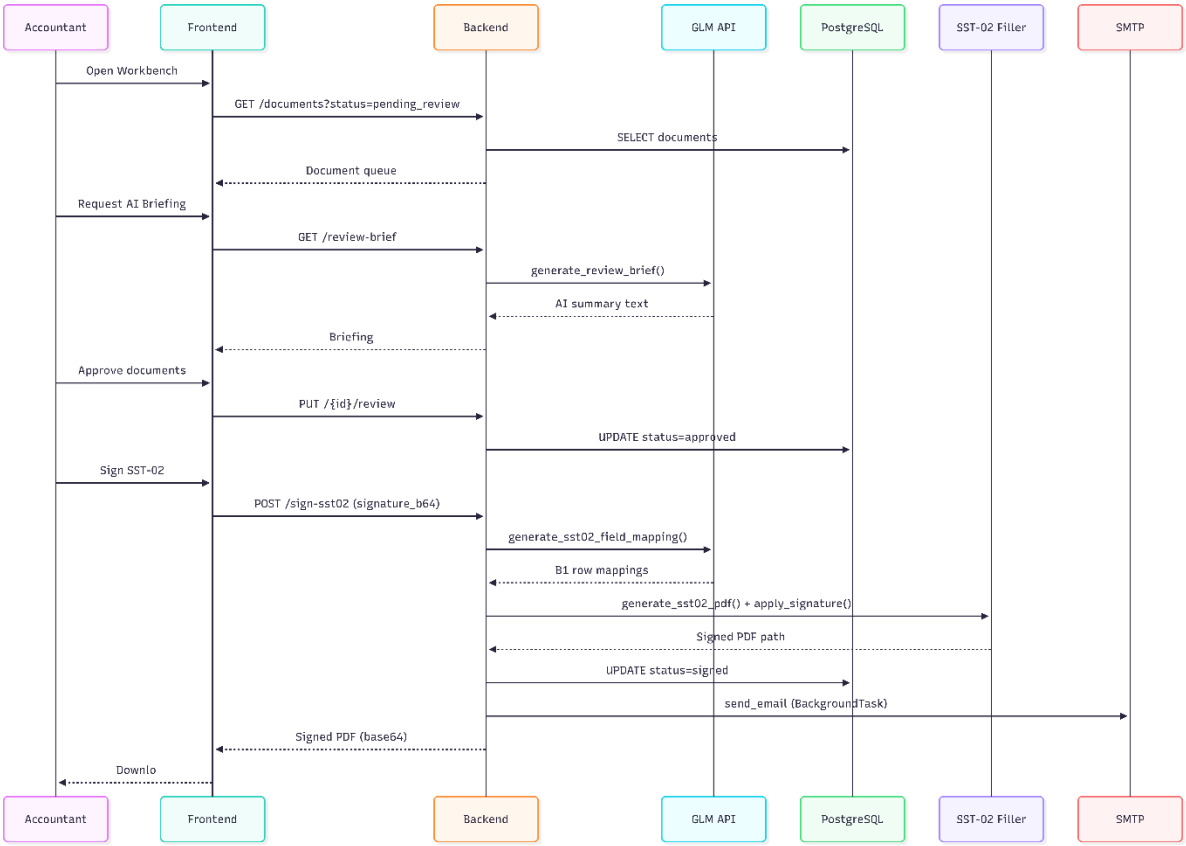

Figure 2: Accountant Review & SST-02 Signing Flow Sequence Diagram

2.2 Technological Stack

2.2.1 Fronted

1. React 19 + TypeScript: type-safe component architecture with strict typing.
2. Vite 8: fast development server with HMR.
3. TailwindCSS 4: utility-first responsive styling.
4. React Router 7: role-based navigation.
5. TanStack React Query: data fetching and caching.
6. Axios: HTTP client with JWT interceptor.
7. Lucide React: icon library.
8. Canvas API: digital signature capture.

2.2.2 Backend

1. FastAPI (Python 3.10): high-performance async web framework with automatic OpenAPI docs
2. SQLAlchemy 2.0: ORM for PostgreSQL
3. Uvicorn: ASGI server
4. Z.AI API Client (Anthropic-compatible SDK): function calling for Tax Agent and Orchestrator
5. PaddleOCR: on-premise OCR for Chinese/English/Malay text. pdfplumber: PDF text extraction.
6. pypdf + ReportLab: SST-02 PDF filling and signature overlay.

7. fastapi-mail: async email with SMTP/STARTTLS
8. python-jose + bcrypt: JWT auth and password hashing
9. Tenacity: retry with exponential backoff for LLM API calls

2.2.3 Database

1. PostgreSQL 16: relational database for users, documents, agent results, and audit trail. Hosted in Docker container via docker-compose with persistent volume. Connected via psycopg2-binary.

2.2.4 Cloud/Deployment

1. Docker Compose: orchestrates PostgreSQL 16 container with persistent volume (postgres_data)
2. Local development: Frontend on Vite dev server (port 5173), Backend on Uvicorn (port 8000), with Vite proxy for API routing.
3. Production-ready: Docker Compose can be extended to containerize FastAPI backend and serve built React frontend via Nginx

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)

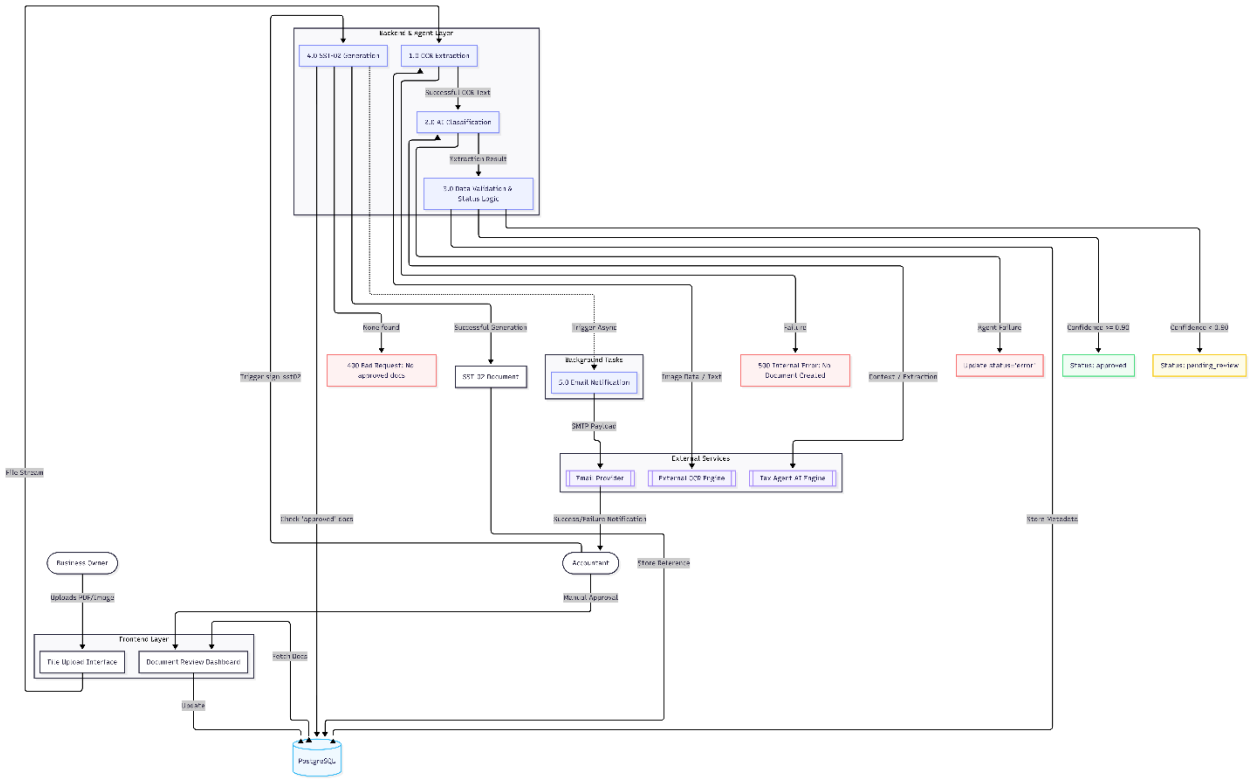

Figure 3: Data Flow Diagram (DFD)

2.3.2 Normalized Database Schema

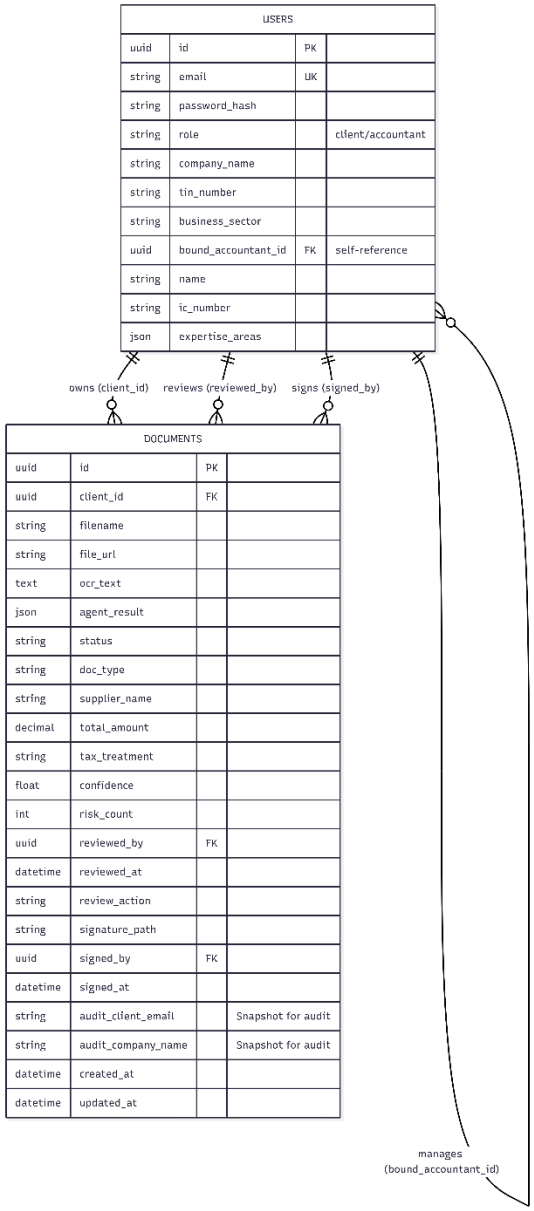

Figure 4: ERD

Normalization analysis

1NF: All columns contain atomic values. agent_result (JSON) and expertise_areas (JSON) are denormalized by design — they are read-heavy snapshots always read/written as whole units, never queried by sub-fields in SQL.

2NF: All non-key attributes depend on the entire primary key. No partial dependencies.

3NF: No transitive dependencies. client_email and company_name on the Document table are conscious violations for audit integrity, historical documents retain values at time of upload even if the user updates their profile.

3. Functional Requirements & Scope

3.1 Minimum Viable Product:

No	Features	Description
1	Intelligent Onboarding & Binding	Profile Setup: Users configure company profiles (TIN, SST ID, sector). Exclusive Binding: During registration, business owners can enter a specific Accountant's Email to create a dedicated link. If left blank, the system auto-matches an accountant based on the user's business sector and expertise areas.
2	AI Receipt Upload & Classification	Business owners upload receipt images (JPG/PNG) or PDFs via drag-and-drop. PaddleOCR extracts text, then the GLM-powered Tax Agent executes a multi-step tool-calling pipeline (SST rule lookup, SST number validation, tax calculation with cross-verification, human review flagging, tax advice generation) to produce a structured classification with confidence score, tax treatment, risk flags, and reasoning
3	Business Owner Dashboard & Insights	Tax Insights: AI analyzes spending patterns to provide real-time tax liability estimates and tax-saving suggestions.

		Draft Visibility: Owners can view the Live Draft of their tax report form (SST-02 form) while waiting for the accountant's review and signature, ensuring transparency.
4	Accountant Review Workbench	Accountants receive an AI-generated briefing summarizing all documents. They review documents in a priority queue (high-risk first), edit AI-extracted fields, approve or reject individual receipts, and tax agent can auto-approve high-confidence receipts (≥90% confidence, 0 risk flags).
5	Tax report form (SST-02 Form) Generation & Digital Signing	After all documents are approved, the accountant signs using a canvas-based digital signature pad. The system uses AI to map documents to SST-02 B1 rows, fills the official JKDM PDF template (AcroForm), overlays the signature, and emails the signed PDF to the business owner with an AI-personalized summary.
6	AI Workflow Orchestrator	A GLM-powered orchestrator monitors the 7-stage compliance pipeline and provides adaptive recommendations. It considers document states, risk percentages, confidence averages, filing deadlines, and configurable risk thresholds.
7	AI Copilot Chat Interface	Both owners and accountants can ask natural language questions about their tax data. The chatbot retrieves current month's documents, builds a context-enriched prompt, and returns evidence-based answers referencing specific receipts and Malaysian tax rules.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Scalability	Handle concurrent uploads near SST filing deadlines (last week of month following each bi-monthly period).	FastAPI async handlers. OCR lazy-loaded singleton. Batch analysis cached 5 min (TTL=300). SQLAlchemy connection pooling.
Reliability	No silent failures. OCR/GLM errors captured and reported, not orphaned.	Documents stored with status="error" on failure. Tenacity retry (3 attempts, exponential backoff). Rule-based fallback for orchestrator. Non-blocking email via BackgroundTasks.
Maintainability	Each module separated by domain concern. Clear layered architecture.	Backend: agents/, services/, routers/, models/, schemas/. Frontend: pages/ (Owner+Accountant), components/, api/, context/, lib/.
Token Latency	GLM calls within 180s. Single-doc classification under 15s at p95. Batch analysis under 30s.	Anthropic client timeout=180s. Temperature=0.1 for deterministic output. Max 8 steps/doc, 5 steps/orchestrator. Batch cache prevents redundant calls.

Cost Efficiency	Minimize token cost per document. No redundant GLM calls for same filing period.	Batch analysis cached with 5-min TTL. SST rule lookup is local keyword-match (not LLM). Auto-approval at ≥90% skips review overhead. Rule-based fallback avoids GLM costs when simple logic suffices.

3.3 Out of Scope / Future Enhancements

- MySST Portal Integration: Direct submission of SST-02 to JKDM MySST portal via API
- E-Invoice Generation: MyInvois-compliant e-invoice generation for B2B transactions
- Multi-Company Support: Single-owner account managing multiple companies with separate SST registrations
- Mobile Application: Native iOS/Android app for receipt capture with camera integration
- Historical Trend Analysis: Cross-period comparison of tax liabilities and spending patterns
- Accounting Software Integration: Import from SQL Account, AutoCount, or Xero
- EPF/SOCSO/EIS Automation: Employee statutory deduction computation and filing
- RBAC & Comprehensive Audit Log: Granular permissions and full regulatory audit trail

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation:

4.1.1 Grayscale Rollout & A/B Testing

Phase 1 (5%): New AI classification model or prompt released to 5% of users (e.g., one business sector). Monitor: classification accuracy, false positive rate for risk flags, SST validation error rate. Pause if metrics degrade >5%.

Phase 2 (25%): After 48 hours of stability, expand to 25% across multiple sectors.

Phase 3 (100%): Full deployment after confirming stability. A/B test comparing prompt strategies (e.g., mandatory tool sequence vs. adaptive tool order) on accuracy, token usage, and processing time.

4.1.2 Emergency Rollback (ER) & Golden Release

Golden Release: Last stable version tagged in git with versioned Docker images.

Rollback triggers:

1) GLM API >5% 5xx errors over 5 min → auto-fallback to rule-based recommendations;

2) Backend unresponsive → restart from Golden Release image within 2 min;

3) Prompt causes hallucinations (accountant rejection >20%) → revert git commit and redeploy.

4.1.3 Priority Matrix

Priority	Condition	Action
P1 Critical	GLM API rate limit >10% 429 errors in 5 min OR data corruption detected	Activate rule-based fallback. Queue new uploads. Notify dev team. Roll back to Golden Release if persistent.
P2 High	OCR returns empty text for >10% uploads OR accountant rejection rate >20%	Switch to pdfplumber-only extraction. Alert dev team. Investigate OCR model.
P3 Medium	Batch analysis incorrect totals OR email delivery failure >5%	Disable batch cache (force fresh). Switch to fallback SMTP relay. Investigate next business day.
P4 Low	UI rendering issues OR slow page load >5s	Log for next sprint. No immediate action.

4.2 Monitoring

Component	Metric	Tool	Alert Threshold
GLM API	Response time, error rate, token usage	Application logging in tax_agent.py	>180s response OR >5% error rate
OCR Engine	Processing time, char count, empty result rate	Console logging in ocr_service.py	>30s processing OR >10% empty results
PostgreSQL	Connection pool, query latency, disk usage	SQLAlchemy logging + pg_stat	>80% pool OR queries >2s
FastAPI	Request rate, response time, error rates	/health endpoint + Uvicorn logs	>5s p95 OR >1% 5xx rate
Document Pipeline	Status distribution, avg confidence, risk flags	Query documents table metrics	>50% error status OR avg conf <0.5
Email	Send success/failure rate	fastapi-mail error logging	>5% failure rate

4.3 Assumptions

a) Users have stable internet connections during receipt upload and SST-02 download operations.

b) Business owners have digital copies (photos/scans) of receipts. TaxMate does not handle physical scanning hardware.

c) Accountants have desktop browsers with pointer input for the canvas-based digital signature pad.

d) Z.AI GLM API (api.ilmu.ai) maintains >99% uptime during evaluation. If unavailable, system degrades to rule-based recommendations.

e) Receipts are primarily in English, Malay, or Chinese — PaddleOCR is configured with lang="ch" for all three.

f) The official SST-02 PDF template AcroForm field names do not change during the hackathon period.

g) The configured SMTP server is operational and allows programmatic email sending.

4.4 External Dependencies

Tools	Purpose	Risks	Mitigation
Z.AI GLM API (ilmu-glm-5.1)	Core LLM reasoning: tax classification, workflow orchestration, batch analysis, chat, email personalization	HIGH — API rate limits/downtime breaks core product. Hallucinations could cause wrong tax treatment	Rule-based fallback. Cached results with error flags. Retry with exponential backoff. Temperature=0.1 for deterministic output.
PaddleOCR	On-premises OCR for receipt images (JPG/PNG)	MEDIUM — Low-quality images may fail extraction.	PIL RGB conversion. Confidence threshold 0.3. Clear error message for empty results.

PostgreSQL 16	Persistent storage for all data	MEDIUM — Unavailability blocks all operations.	Docker persistent volume. SQLAlchemy connection pooling with auto-reconnect.
SMTP Server	Email signed SST-02 PDFs to owners	LOW — Failure doesn't block signing workflow.	Non-blocking via BackgroundTasks. PDF still available for download.
JKDM SST-02 Template	Official government PDF form for filling	LOW — Template changes could break field mapping.	Version-controlled in templates/. pypdf try/except for graceful degradation.

5. Project Management & Team Contributions

5.1 Project Timeline

Day	Activity	Justification
1-2	Project Planning & Architecture	PRD documentation, Define scope, identify Malaysian tax pain points, design system architecture (agent patterns, data flows, DB schema), select tech stack, set up repo and Docker Compose
3-4	Backend Development + GLM Integration	FastAPI structure, DB models, auth (JWT+bcrypt), OCR service, Tax

		Agent with 5 tool definitions and multi-step function calling, structured JSON parsing
5-6	Advanced Agent Features	Workflow Orchestrator with adaptive rules, batch analysis agent, review briefing, AI email, chat endpoint, SST-02 PDF filling (pypdf AcroForm), digital signature overlay (ReportLab), tax insight, auto approve
7-8	Frontend Development	React SPA with Owner interface (Dashboard, Upload, Documents, Tax Insights, Downloads, Chatbot, Profile) and Accountant interface (Client List, Workbench with queue-based review, digital signature pad, Efficiency Summary, Profile)
9-10	Testing, Documentation & Demo	QTAD documentation, End-to-end pipeline testing, edge case fixes (OCR failures, invalid SST numbers, rate limits), SAD documentation, demo script with realistic Malaysian receipt data

5.2 Team Members Role

Member	Role	Responsibilities
Yap Li Shan	Backend Lead & AI Agent Architect	a) FastAPI application architecture b) SQLAlchemy schema design c) Z.AI GLM API integration d) Tax Agent tool definitions and execution logic e) Workflow Orchestrator adaptive rules f) Batch analysis agent

		g) OCR service (PaddleOCR) h) SST-02 PDF template filling (pypdf AcroForm) i) Digital signature overlay (ReportLab) j) Email service k) Authentication system (JWT + bcrypt) l) Logic for automatically matching accountant
Chan Min Huey	Frontend Lead & Product Manager	a) React 19 + TypeScript SPA architecture b) Owner and Accountant interface development c) SignaturePad canvas component d) AI Chatbot floating widget e) Axios API client with JWT interceptor f) Responsive TailwindCSS styling g) Demo preparation h) Documentation coordination i) Pitch deck design

5.3 Recommendations

- Redis Cache for Batch Analysis: Currently caught in memory with Python dict. For production with multiple instances, migrate to Redis with proper TTL and cross-instance invalidation.
- Async LLM Pipeline: Currently synchronous, blocking workers 10-30s per document. Implement Celery + Redis task queue to return document ID immediately and poll for results.
- Prompt Versioning: Store prompt templates in configuration files or database for A/B testing and rapid rollback without code deployment.